

AMAZON BRAZIL E-COMMERCE DATA ANALYSIS – MILESTONE 2

Complete SQL Analysis, Outputs & Insights

Meeteshh Kumar Dey

PROJECT OVERVIEW

- Objective: Analyze Amazon Brazil E-Commerce data using SQL and identify business insights related to sales, customers, products, payments, and revenue trends.
- Tools: PostgreSQL, SQL, Excel, pgAdmin 4.
- Main tables: Customers, Products, Orders, Order Items, Order Payments, Sellers.

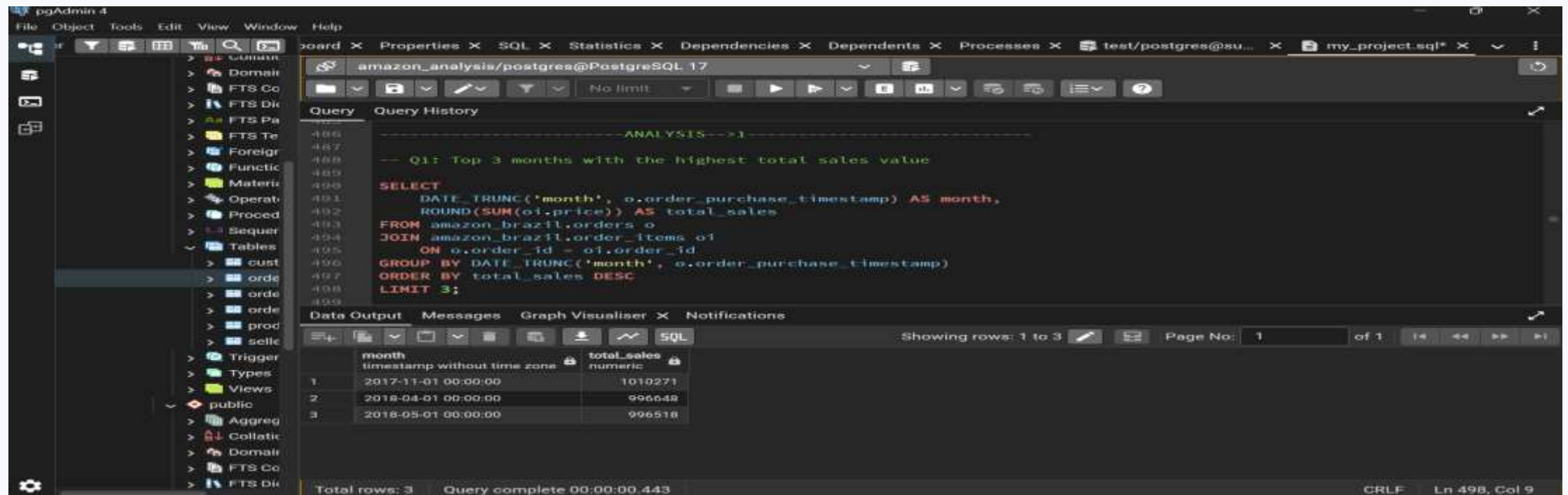
ANALYSIS-I

Questions, SQL Outputs and Insights

Q1. To identify seasonal sales patterns, determine the top

Question

Q1. To identify seasonal sales patterns, determine the top



The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including a 'public' schema with various tables and views. The main window shows a SQL query titled 'Q1: Top 3 months with the highest total sales value'. The query is as follows:

```
-- Q1: Top 3 months with the highest total sales value
SELECT
    DATE_TRUNC('month', o.order_purchase_timestamp) AS month,
    ROUND(SUM oi.price)) AS total_sales
FROM amazon_brazil.orders o
JOIN amazon_brazil.order_items oi
    ON o.order_id = oi.order_id
GROUP BY DATE_TRUNC('month', o.order_purchase_timestamp)
ORDER BY total_sales DESC
LIMIT 3;
```

The query results are displayed in a table with the following data:

month	total_sales
2017-11-01 00:00:00	1010271
2018-04-01 00:00:00	996548
2018-05-01 00:00:00	996518

The status bar at the bottom indicates 'Total rows: 3' and 'Query complete 00:00:00.443'.

KEY INSIGHT

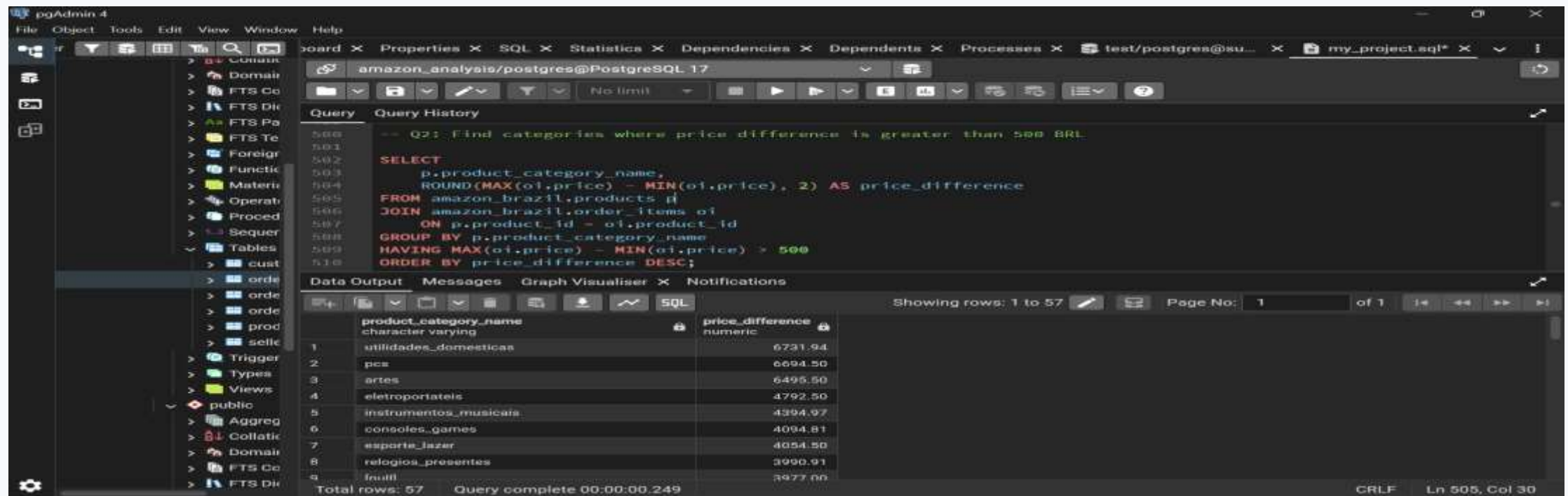
Q1. To identify seasonal sales patterns, determine the top

The analysis identifies the three months with the highest total sales value, helping Amazon understand its strongest sales periods and plan inventory and marketing activities accordingly.

Q2. Amazon India is interested in project categories with significant price variations. Find categories

Question

Q2. Amazon India is interested in project categories with significant price variations. Find categories where the difference between the maximum and minimum product prices is greater than 500 BRL.



The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including a 'public' schema with various tables and views. The main window shows a SQL query being executed. The query is designed to find product categories where the price difference between the maximum and minimum product prices is greater than 500 BRL. The results are displayed in a table with two columns: 'product_category_name' and 'price_difference'.

```
-- Q2: Find categories where price difference is greater than 500 BRL
SELECT
    p.product_category_name,
    ROUND(MAX(oi.price) - MIN(oi.price), 2) AS price_difference
FROM amazon_brazil.products p
JOIN amazon_brazil.order_items oi
ON p.product_id = oi.product_id
GROUP BY p.product_category_name
HAVING MAX(oi.price) - MIN(oi.price) > 500
ORDER BY price_difference DESC;
```

	product_category_name	price_difference
1	utilidades_domesticas	6731.94
2	pcas	6694.50
3	artes	6495.50
4	eletroportateis	4792.50
5	instrumentos_musicais	4394.97
6	consoles_games	4094.81
7	esporte_lazer	4054.50
8	relogios_presentes	3990.91
9	fruits	3977.00

Total rows: 57 Query complete 00:00:00.249

KEY INSIGHT

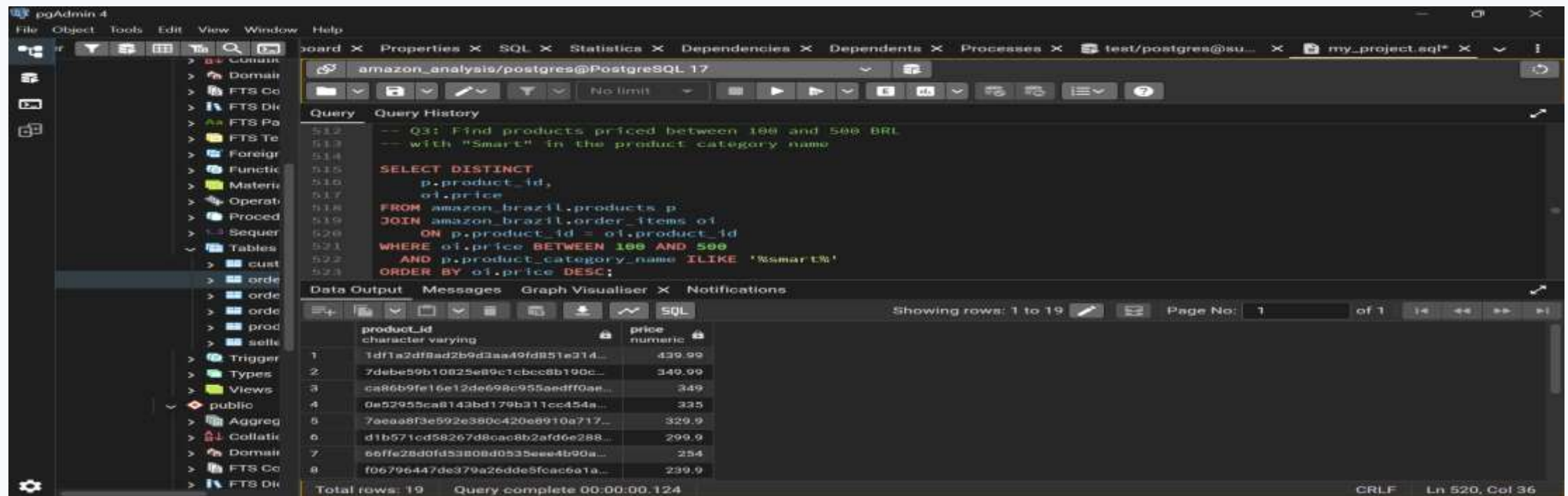
Q2. Amazon India is interested in project categories with significant price variations. Find categories where the difference between the maximum and minimum product prices is greater than 500 BRL.

The analysis identifies product categories with significant price variations, where the difference between the maximum and minimum product prices is greater than 500 BRL. This helps Amazon understand categories with wide price ranges and potential pricing opportunities.

Q3. Amazon wants to identify which payment types have the most consistent transaction amount

Question

Q3. Amazon wants to identify which payment types have the most consistent transaction amounts. Identifies payment types with the least standard deviation, sorted from smallest to largest.



The screenshot shows the pgAdmin 4 interface with a SQL query executed. The query is designed to find products priced between 100 and 500 BRL with "Smart" in the product category name. The results are displayed in a table with two columns: product_id and price.

```
-- Q3: Find products priced between 100 and 500 BRL
-- with "Smart" in the product category name

SELECT DISTINCT
    p.product_id,
    oi.price
FROM amazon_brazil.products p
JOIN amazon_brazil.order_items oi
ON p.product_id = oi.product_id
WHERE oi.price BETWEEN 100 AND 500
AND p.product_category_name ILIKE '%smart%'
ORDER BY oi.price DESC;
```

	product_id	price
1.	1df1a2df8ad2b9d3aa49fd851e314...	439.99
2.	7debe59b10025e89c1cbcc8b190c...	340.99
3.	ca86b9fe16e12de698c955aedff0a...	349
4.	0e52955ca8143bd179b311cc454a...	325
5.	7aeaa8f3e592e380c420e8910a717...	329.9
6.	d1b571cd58267d8cac8b2afd6e288...	299.9
7.	66ffe28d0fd53808d0535ee4b90a...	254
8.	f06796447de379a26dde5fcac6a1a...	239.9

Total rows: 19 Query complete 00:00:00.124 CRLF Ln 520, Col 36

KEY INSIGHT

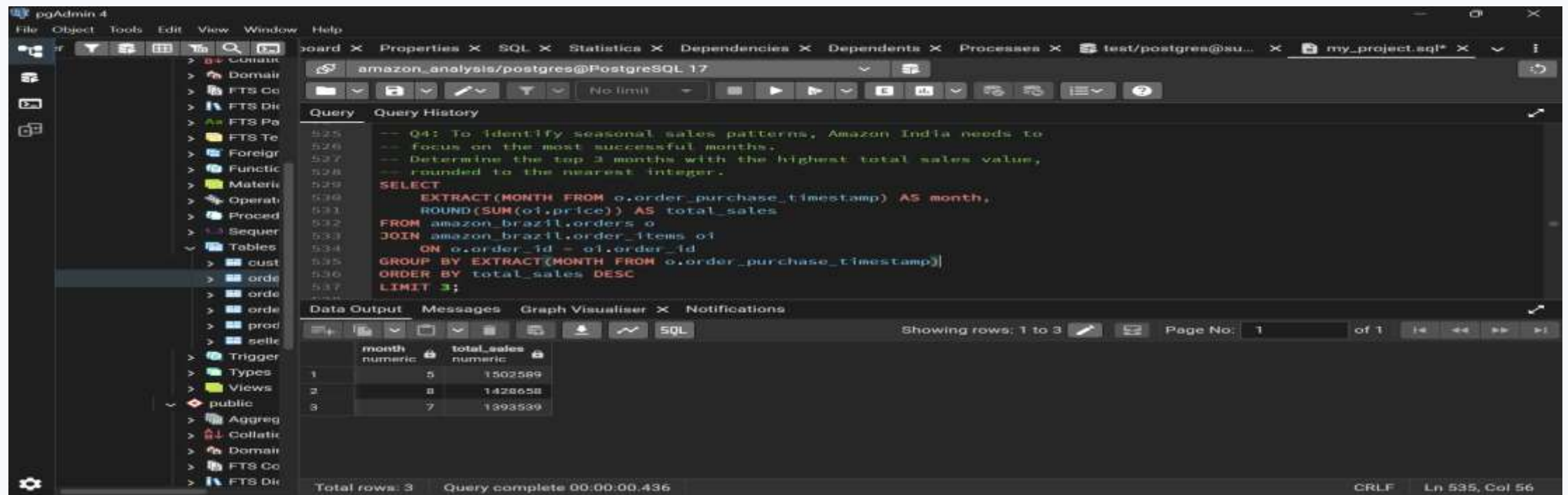
Q3. Amazon wants to identify which payment types have the most consistent transaction amounts. Identifies payment types with the least standard deviation, sorted from smallest to largest.

Insight is retained in the original submitted project document.

Q4. To identify seasonal sales patterns, Amazon India needs to focus on the most successful months

Question

Q4. To identify seasonal sales patterns, Amazon India needs to focus on the most successful months. Determine the top 3 months with the highest total sales value, rounded to the nearest integer.



The screenshot shows the pgAdmin 4 interface with a SQL query executed. The query is designed to identify the top 3 months with the highest total sales value from the Amazon Brazil orders and order items tables. The results are displayed in a table with 3 rows and 2 columns: month and total_sales.

```
-- Q4: To identify seasonal sales patterns, Amazon India needs to
-- focus on the most successful months.
-- Determine the top 3 months with the highest total sales value,
-- rounded to the nearest integer.
SELECT
    EXTRACT(MONTH FROM o.order_purchase_timestamp) AS month,
    ROUND(SUM(o1.price)) AS total_sales
FROM amazon_brazil.orders o
JOIN amazon_brazil.order_items o1
    ON o.order_id = o1.order_id
GROUP BY EXTRACT(MONTH FROM o.order_purchase_timestamp)
ORDER BY total_sales DESC
LIMIT 3;
```

month	total_sales
1	1502589
2	1428658
3	1393539

Total rows: 3 Query complete 00:00:00.436 CRLF Ln 535, Col 56

KEY INSIGHT

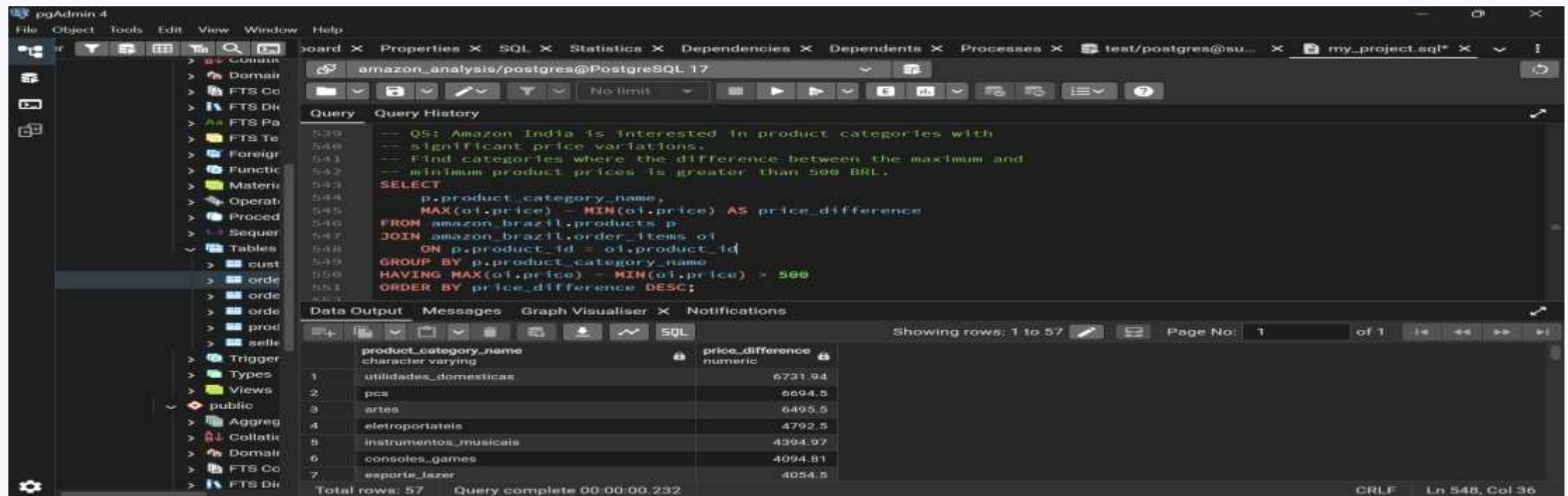
Q4. To identify seasonal sales patterns, Amazon India needs to focus on the most successful months. Determine the top 3 months with the highest total sales value, rounded to the nearest integer.

The analysis identifies May, August, and July as the three months with the highest total sales value, indicating strong sales performance during these periods.

Q5. Amazon India is interested in product categories with significant price variations. Find categories

Question

Q5. Amazon India is interested in product categories with significant price variations. Find categories where the difference between the maximum and minimum product prices is greater than 500 BRL.



The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including a 'public' schema with various tables. The main window shows a SQL query being executed. The query is as follows:

```
-- Q5: Amazon India is interested in product categories with
-- significant price variations.
-- Find categories where the difference between the maximum and
-- minimum product prices is greater than 500 BRL.
SELECT
    p.product_category_name,
    MAX(oi.price) - MIN(oi.price) AS price_difference
FROM amazon_brazil.products p
JOIN amazon_brazil.order_items oi
    ON p.product_id = oi.product_id
GROUP BY p.product_category_name
HAVING MAX(oi.price) - MIN(oi.price) > 500
ORDER BY price_difference DESC;
```

The 'Data Output' tab shows the results of the query. The table has two columns: 'product_category_name' and 'price_difference'. The results are as follows:

product_category_name	price_difference
utilidades_domesticas	6731.94
pcs	6694.5
artes	6495.5
eletroportateis	4792.5
instrumentos_musicais	4394.97
consoles_games	4094.81
esporte_lazer	4054.5

The status bar at the bottom indicates 'Total rows: 57' and 'Query complete 00:00:00.232'.

KEY INSIGHT

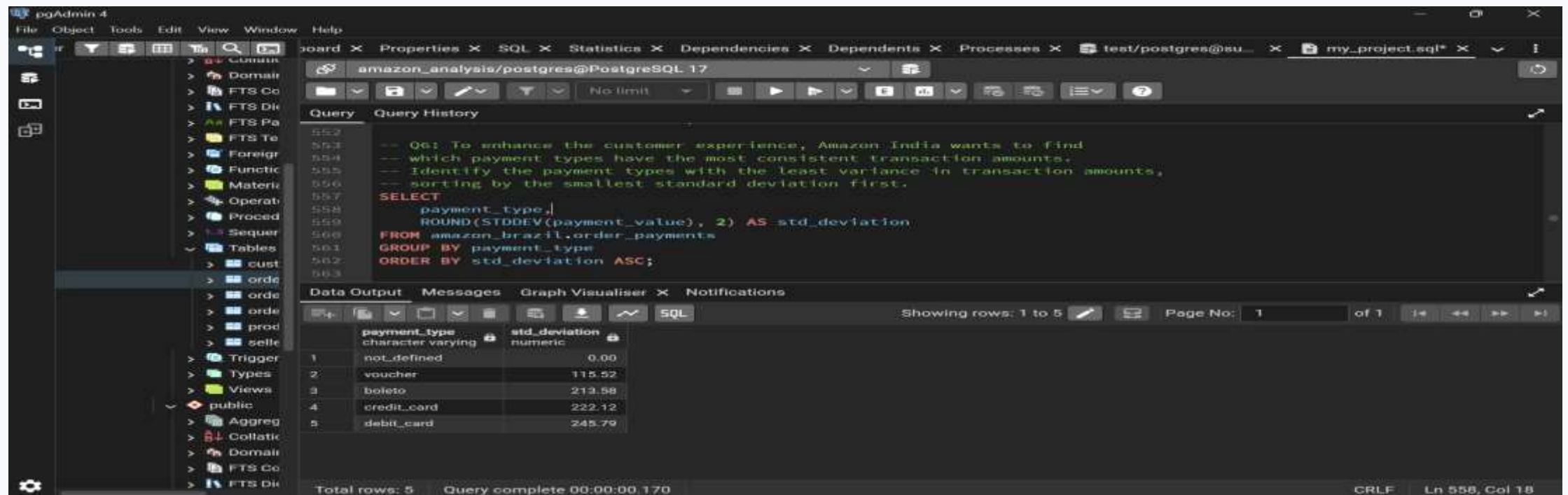
Q5. Amazon India is interested in product categories with significant price variations. Find categories where the difference between the maximum and minimum product prices is greater than 500 BRL.

The analysis shows the most popular payment types across low, medium, and high order value segments. This helps Amazon understand customer payment preferences across different spending levels.

Q6. To enhance the customer experience, Amazon India wants to find which payment types have

Question

Q6. To enhance the customer experience, Amazon India wants to find which payment types have the most consistent transaction amounts. Identify the payment types with the least variance in transaction amounts, sorting by the smallest standard deviation first.



The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including a 'public' schema. The main window shows a SQL query being executed. The query is as follows:

```
-- Q6: To enhance the customer experience, Amazon India wants to find
-- which payment types have the most consistent transaction amounts.
-- Identify the payment types with the least variance in transaction amounts,
-- sorting by the smallest standard deviation first.
SELECT
    payment_type,
    ROUND(STDDEV(payment_value), 2) AS std_deviation
FROM amazon_brazil.order_payments
GROUP BY payment_type
ORDER BY std_deviation ASC;
```

The results are displayed in a table with the following data:

	payment_type	std_deviation
1	not_defined	0.00
2	voucher	115.52
3	boleto	213.58
4	credit_card	222.12
5	debit_card	245.79

The status bar at the bottom indicates 'Total rows: 5' and 'Query complete 00:00:00.170'.

KEY INSIGHT

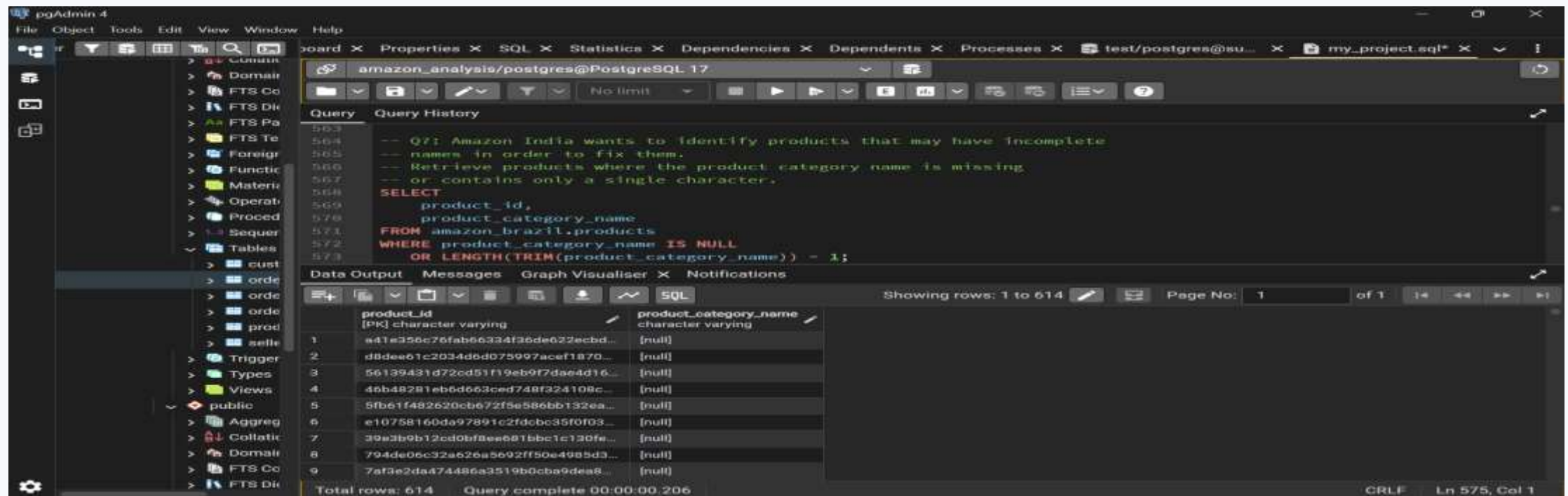
Q6. To enhance the customer experience, Amazon India wants to find which payment types have the most consistent transaction amounts. Identify the payment types with the least variance in transaction amounts, sorting by the smallest standard deviation first.

The analysis identifies the payment types with the most consistent transaction amounts by comparing their standard deviation. Payment types with the smallest standard deviation show less variation in transaction values, indicating more consistent customer payment behavior.

Q7. Amazon India wants to identify products that may have incomplete name in order to fix it from

Question

Q7. Amazon India wants to identify products that may have incomplete name in order to fix it from their end. Retrieve the list of products where the product category name is missing or contains only a single character.



The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including a 'public' schema with various tables. The main window shows a SQL query editor with the following query:

```
-- Q7: Amazon India wants to identify products that may have incomplete
-- names in order to fix them.
-- Retrieve products where the product category name is missing
-- or contains only a single character.
SELECT
    product_id,
    product_category_name
FROM amazon_brazil.products
WHERE product_category_name IS NULL
OR LENGTH(TRIM(product_category_name)) = 1;
```

The query results are displayed in a table with 2 columns: 'product_id' and 'product_category_name'. The table shows 9 rows of data, all with null values in the 'product_category_name' column.

product_id	product_category_name
1	[null]
2	[null]
3	[null]
4	[null]
5	[null]
6	[null]
7	[null]
8	[null]
9	[null]

The status bar at the bottom indicates 'Total rows: 614' and 'Query complete 00:00:00.206'.

KEY INSIGHT

Q7. Amazon India wants to identify products that may have incomplete name in order to fix it from their end. Retrieve the list of products where the product category name is missing or contains only a single character.

The analysis identifies 614 products with missing or incomplete product category names. These records should be reviewed and corrected to improve data quality, product classification, and the accuracy of business analysis.

ANALYSIS-II

Questions, SQL Outputs and Insights

Q1. Amazon India wants to understand which payment types are most popular across different

Question

Q1. Amazon India wants to understand which payment types are most popular across different order value segments (e.g., low, medium, high). Segment order values into three ranges: orders less than 200 BRL, between 200 and 1000 BRL, and over 1000 BRL. Calculate the count of each payment type within these ranges and display the results in descending order of count.

The screenshot shows the pgAdmin 4 interface with a SQL query editor. The query is as follows:

```

-- Q1: Amazon India wants to understand which payment types are most
-- popular across different order value segments.
-- Segment orders into:
-- Low: less than 200 BRL
-- Medium: between 200 and 1000 BRL
-- High: over 1000 BRL
-- Calculate the count of each payment type within these segments.
WITH order_values AS (
    SELECT
        order_id,
        SUM(payment_value) AS total_order_value
    FROM amazon_brazil.order_payments
    GROUP BY order_id
)
SELECT
    CASE
        WHEN ov.total_order_value < 200 THEN 'Low'
        WHEN ov.total_order_value BETWEEN 200 AND 1000 THEN 'Medium'
        ELSE 'High'
    END AS order_value_segment,
    op.payment_type,
    COUNT(*) AS count
FROM order_values ov

```

The query is executed, and the results are shown in the Data Output tab.

The screenshot shows the pgAdmin 4 interface with the same SQL query as the previous screenshot. The query is executed, and the results are shown in the Data Output tab. The results are as follows:

order_value_segment	payment_type	count
Low	credit_card	40135
Low	boleto	10444
Medium	credit_card	15545
Low	voucher	4813
Medium	boleto	3182
Low	debit_card	1287
High	credit_card	915
Medium	voucher	907
Medium	debit_card	227
High	boleto	178
High	voucher	55
High	credit_card	15
Low	notDefined	1

KEY INSIGHT

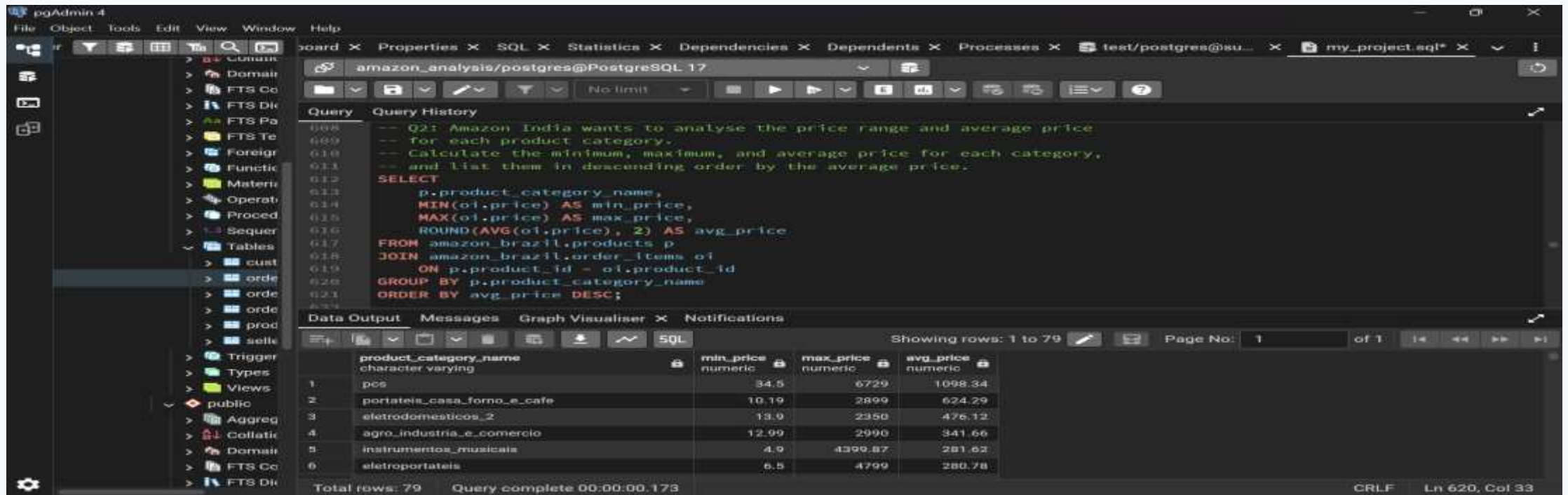
Q1. Amazon India wants to understand which payment types are most popular across different order value segments (e.g., low, medium, high). Segment order values into three ranges: orders less than 200 BRL, between 200 and 1000 BRL, and over 1000 BRL. Calculate the count of each payment type within these ranges and display the results in descending order of count.

The analysis identifies the most popular payment types across different order value segments—Low (< 200 BRL), Medium (200–1000 BRL), and High (> 1000 BRL). This helps Amazon India understand customer payment preferences based on purchase value and identify which payment methods are most commonly used for each spending category.

Q2. Amazon India wants to analyse the price range and average price for each product category.

Question

Q2. Amazon India wants to analyse the price range and average price for each product category. Calculate the minimum, maximum, and average price for each category, and list them in descending order by the average price.



The screenshot shows the pgAdmin 4 interface with a SQL query executed. The query calculates the minimum, maximum, and average price for each product category, ordered by average price in descending order.

```

-- Q2: Amazon India wants to analyse the price range and average price
-- for each product category.
-- Calculate the minimum, maximum, and average price for each category,
-- and list them in descending order by the average price.
SELECT
    p.product_category_name,
    MIN(oi.price) AS min_price,
    MAX(oi.price) AS max_price,
    ROUND(AVG(oi.price), 2) AS avg_price
FROM amazon_brazil.products p
JOIN amazon_brazil.order_items oi
ON p.product_id = oi.product_id
GROUP BY p.product_category_name
ORDER BY avg_price DESC;

```

The results are displayed in a table with 4 columns: product_category_name, min_price, max_price, and avg_price. The table shows 6 rows of data, sorted by average price in descending order.

product_category_name	min_price	max_price	avg_price
pcs	34.5	6729	1098.34
portateis_casa_forno_e_cafe	10.19	2899	624.29
eletrodomesticos_2	13.9	2350	476.12
agro_industria_e_comercio	12.99	2990	341.66
instrumentos_musicais	4.9	4399.87	281.62
eletroportateis	6.5	4799	280.78

Total rows: 79 Query complete 00:00:00.173

KEY INSIGHT

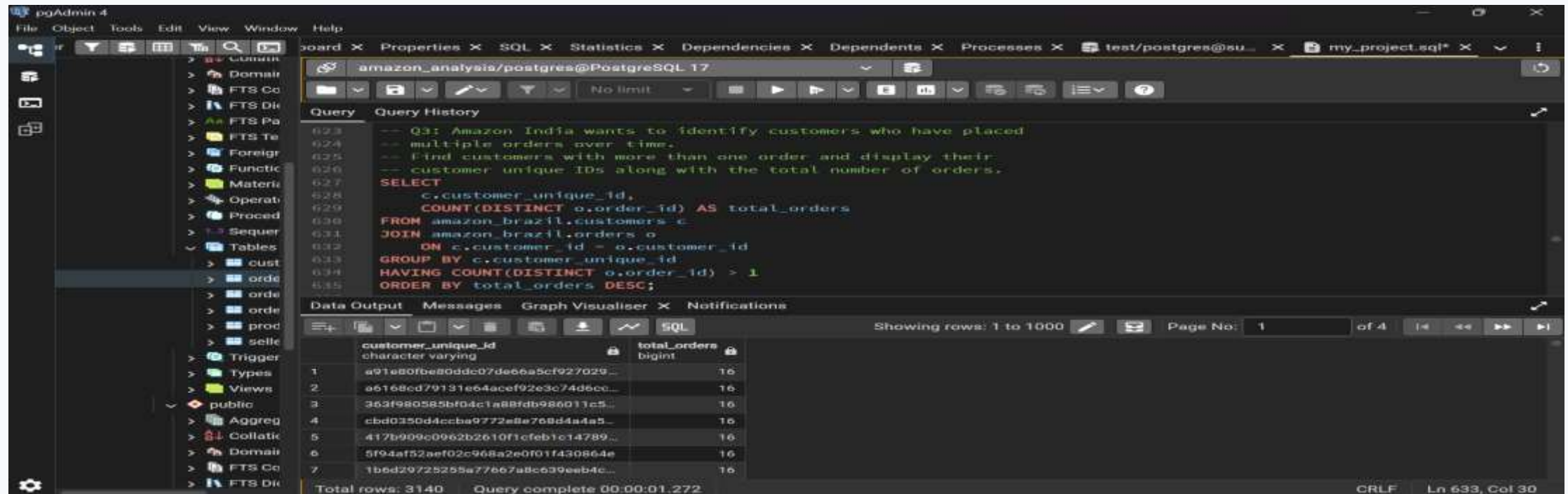
Q2. Amazon India wants to analyse the price range and average price for each product category. Calculate the minimum, maximum, and average price for each category, and list them in descending order by the average price.

The analysis shows that product categories have significant differences in their price ranges and average prices. This helps Amazon India identify high-value product categories, understand pricing variations, and support better pricing and product strategy decisions.

Q3. Amazon India wants to identify the customers who have placed multiple orders over time. F

Question

Q3. Amazon India wants to identify the customers who have placed multiple orders over time. Find all customers with more than one order, and display their customer unique IDs along with the total number of orders they have placed.



The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including a 'public' schema with a 'customers' table. The main window shows a SQL query being executed. The query is as follows:

```

-- Q3: Amazon India wants to identify customers who have placed
-- multiple orders over time.
-- Find customers with more than one order and display their
-- customer unique IDs along with the total number of orders.
SELECT
    c.customer_unique_id,
    COUNT(DISTINCT o.order_id) AS total_orders
FROM amazon_brazil.customers c
JOIN amazon_brazil.orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_unique_id
HAVING COUNT(DISTINCT o.order_id) > 1
ORDER BY total_orders DESC;

```

The 'Data Output' tab shows the results of the query. The results are displayed in a table with two columns: 'customer_unique_id' (character varying) and 'total_orders' (bigint). The table shows 7 rows of data, with the first row having 16 total orders and the last row having 1 total order. The total number of rows is 3140, and the query completed in 00:00:01.272 seconds.

	customer_unique_id	total_orders
1	a91e80f6e80ddc07de66a5cf927029...	16
2	a6168cd79131e64acef92e3c74d6cc...	16
3	363f980585bf04c1a88fdb986011c5...	16
4	cbd0350d4ccba9772e8e768d4a4a5...	16
5	417b909c0962b2610f1cfef1c14789...	16
6	5f94af52aef02c968a2e0f01f430864e...	16
7	1b6d29725255a77667a8c639eeb4c...	16

KEY INSIGHT

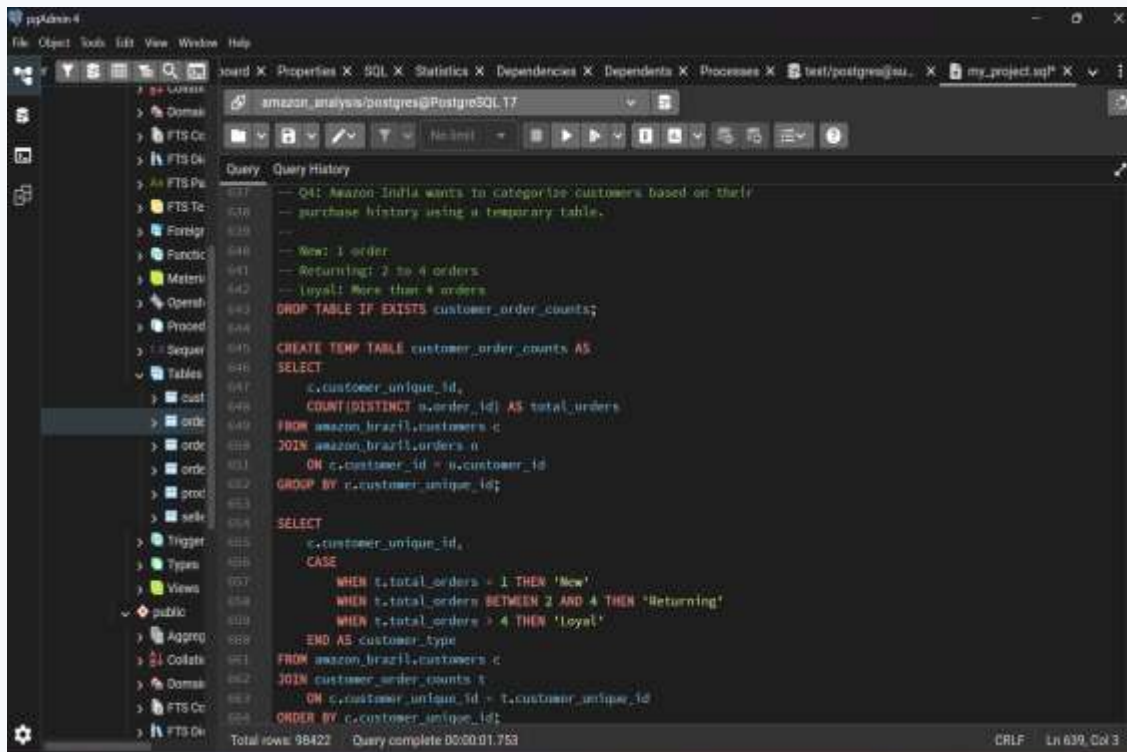
Q3. Amazon India wants to identify the customers who have placed multiple orders over time. Find all customers with more than one order, and display their customer unique IDs along with the total number of orders they have placed.

This analysis identifies repeat customers who have placed more than one order. Customers appearing at the top of the results have the highest number of orders, indicating stronger customer engagement and repeat purchasing behavior. These insights can help Amazon India identify loyal customers and create targeted strategies such as personalized offers, loyalty programs, and retention campaigns.

Q4. Amazon India wants to categorize customers into different types ('New – order qty. = 1' ; 'Re

Question

Q4. Amazon India wants to categorize customers into different types ('New – order qty. = 1' ; 'Returning' – order qty. 2 to 4; 'Loyal' – order qty. >4) based on their purchase history. Use a temporary table to define these categories and join it with the customers table to update and display the customer types.



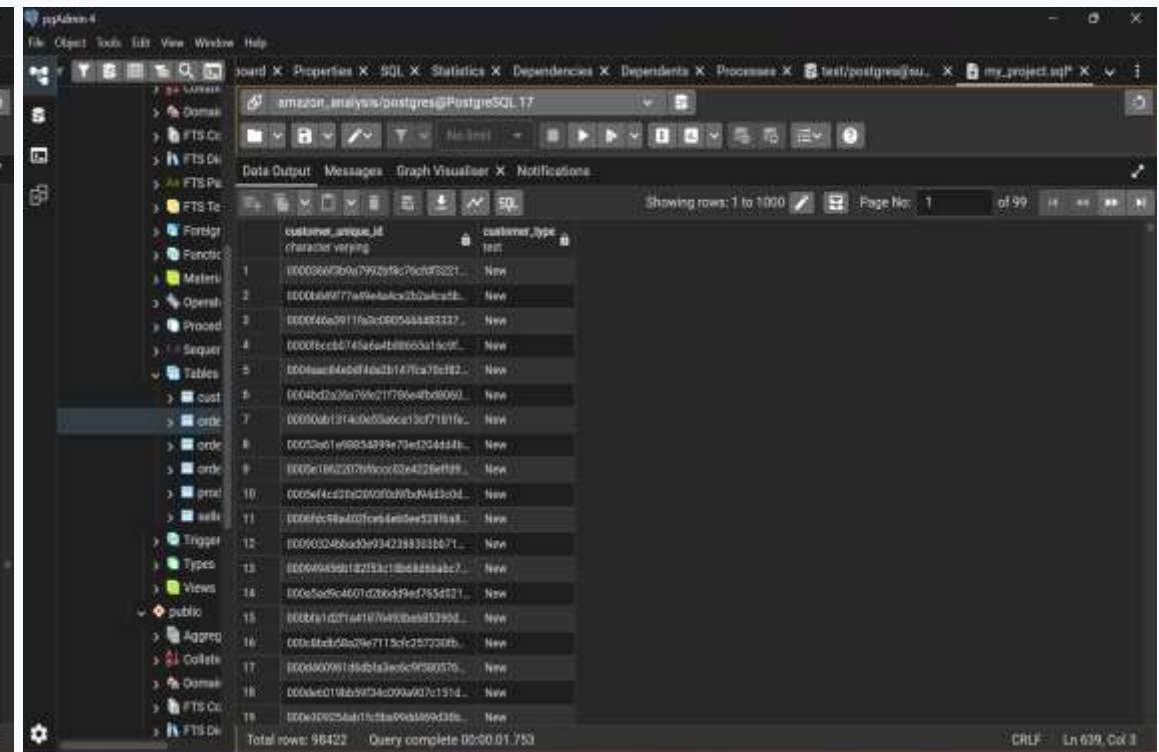
```

-- Q4: Amazon India wants to categorize customers based on their
-- purchase history using a temporary table.
--
-- New: 1 order
-- Returning: 2 to 4 orders
-- Loyal: More than 4 orders
DROP TABLE IF EXISTS customer_order_counts;

CREATE TEMP TABLE customer_order_counts AS
SELECT
    c.customer_unique_id,
    COUNT(DISTINCT o.order_id) AS total_orders
FROM amazon_brazil.customers c
JOIN amazon_brazil.orders o
    ON c.customer_id = o.customer_id
GROUP BY c.customer_unique_id;

SELECT
    c.customer_unique_id,
    CASE
        WHEN t.total_orders = 1 THEN 'New'
        WHEN t.total_orders BETWEEN 2 AND 4 THEN 'Returning'
        WHEN t.total_orders > 4 THEN 'Loyal'
    END AS customer_type
FROM amazon_brazil.customers c
JOIN customer_order_counts t
    ON c.customer_unique_id = t.customer_unique_id
ORDER BY c.customer_unique_id;
  
```

Total rows: 98422 Query complete 00:00:01.753



customer_unique_id	customer_type
00000000000000000000000000000000	New
00000000000000000000000000000000	New
00000000000000000000000000000000	New
00000000000000000000000000000000	New
00000000000000000000000000000000	New
00000000000000000000000000000000	New
00000000000000000000000000000000	New
00000000000000000000000000000000	New
00000000000000000000000000000000	New
00000000000000000000000000000000	New
00000000000000000000000000000000	New
00000000000000000000000000000000	New
00000000000000000000000000000000	New
00000000000000000000000000000000	New
00000000000000000000000000000000	New
00000000000000000000000000000000	New
00000000000000000000000000000000	New
00000000000000000000000000000000	New
00000000000000000000000000000000	New

Total rows: 98422 Query complete 00:00:01.753

KEY INSIGHT

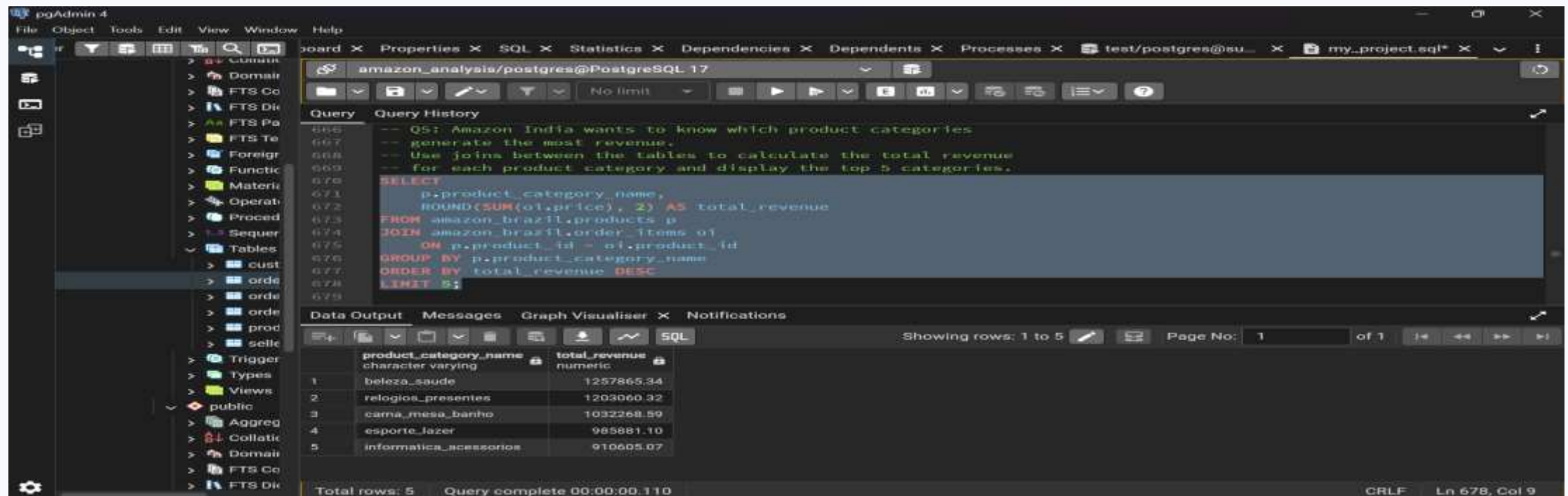
Q4. Amazon India wants to categorize customers into different types ('New – order qty. = 1' ; 'Returning' –order qty. 2 to 4; 'Loyal' – order qty. >4) based on their purchase history. Use a temporary table to define these categories and join it with the customers table to update and display the customer types.

This analysis categorizes customers into New, Returning, and Loyal groups based on their total number of orders. It helps Amazon India understand customer purchasing behavior and identify high-value loyal customers for targeted offers, retention strategies, and loyalty programs.

Q5. Amazon India wants to know which product categories generate the most revenue. Use join

Question

Q5. Amazon India wants to know which product categories generate the most revenue. Use joins between the tables to calculate the total revenue for each product category. Display the top 5 categories.



The screenshot shows the pgAdmin 4 interface. The left sidebar displays the database structure, including a 'public' schema with various tables. The main window shows a SQL query in the 'Query' tab, which is a JOIN query to calculate the total revenue for each product category. The 'Data Output' tab shows the results of the query, displaying the top 5 product categories by total revenue.

```
-- Q5: Amazon India wants to know which product categories
-- generate the most revenue.
-- Use joins between the tables to calculate the total revenue
-- for each product category and display the top 5 categories.

SELECT
    p.product_category_name,
    ROUND(SUM(o1.price), 2) AS total_revenue
FROM amazon_brazil.products p
JOIN amazon_brazil.order_items o1
    ON p.product_id = o1.product_id
GROUP BY p.product_category_name
ORDER BY total_revenue DESC
LIMIT 5;
```

	product_category_name	total_revenue
1	beleza_saude	1257865.34
2	relogios_presentes	1203060.32
3	carna_mesa_banho	1032268.59
4	esporte_lazer	985881.10
5	informatica_acessorios	910605.07

Total rows: 5 Query complete 00:00:00.110 CRLF Ln 678, Col 9

KEY INSIGHT

Q5. Amazon India wants to know which product categories generate the most revenue. Use joins between the tables to calculate the total revenue for each product category. Display the top 5 categories.

The analysis identifies the top 5 product categories generating the highest total revenue. These categories contribute the most to Amazon India's overall sales and can be considered high-value product segments.

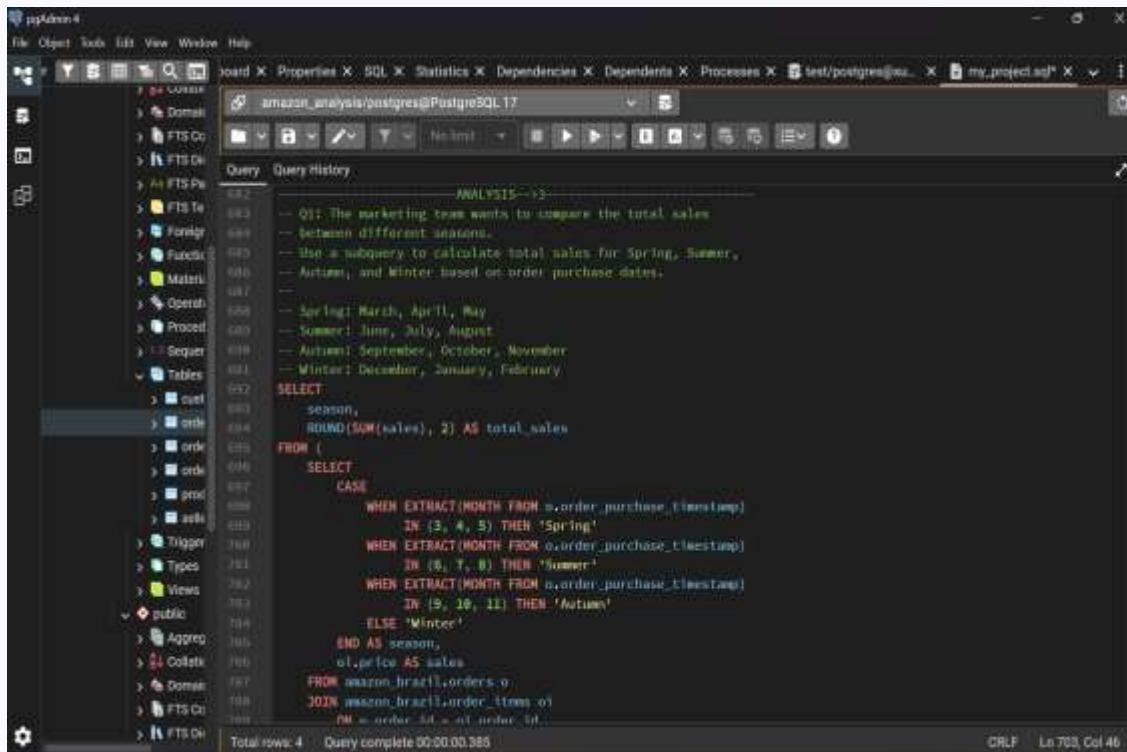
ANALYSIS-III

Questions, SQL Outputs and Insights

Q1. The marketing team wants to compare the total sales between different seasons. Use a subquery

Question

Q1. The marketing team wants to compare the total sales between different seasons. Use a subquery to calculate total sales for each season (Spring, Summer, Autumn, Winter) based on order purchase dates, and display the results. Spring is in the months of March, April and May. Summer is from June to August and Autumn is between September and November and rest months are Winter.

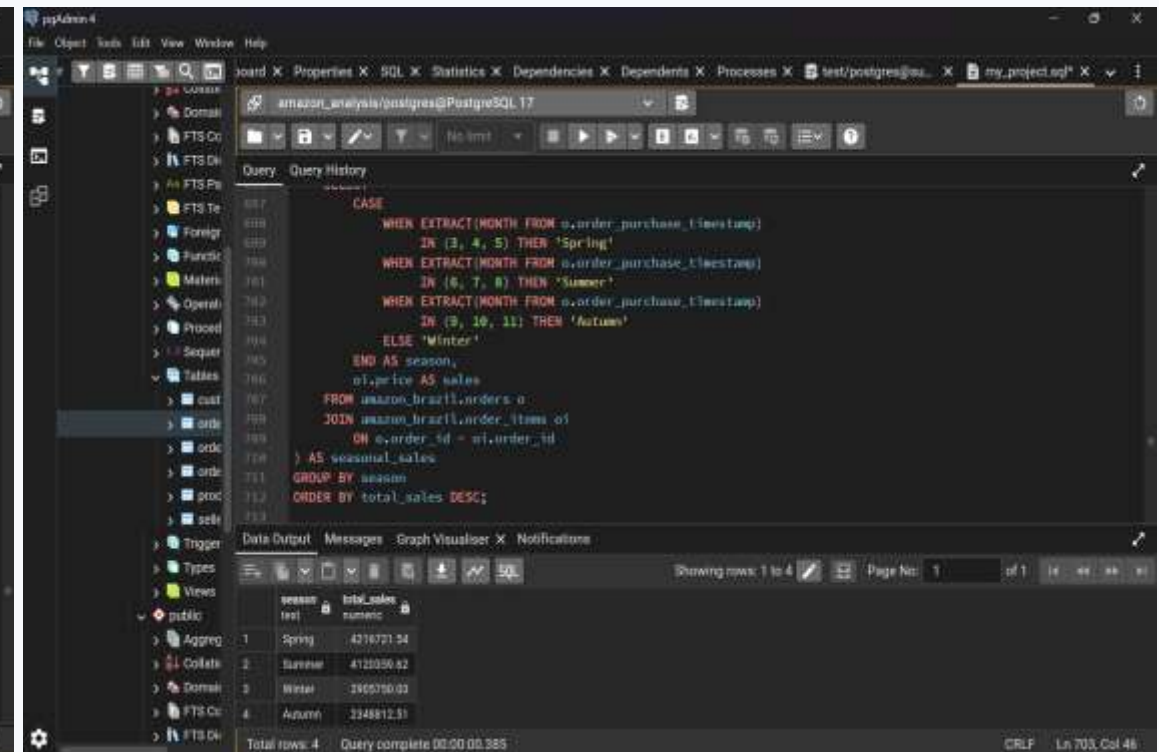


```

-- Q1: The marketing team wants to compare the total sales
-- between different seasons.
-- Use a subquery to calculate total sales for Spring, Summer,
-- Autumn, and Winter based on order purchase dates.
--
-- Spring: March, April, May
-- Summer: June, July, August
-- Autumn: September, October, November
-- Winter: December, January, February

SELECT
    season,
    ROUND(SUM(sales), 2) AS total_sales
FROM (
    SELECT
        CASE
            WHEN EXTRACT(MONTH FROM o.order_purchase_timestamp)
                IN (3, 4, 5) THEN 'Spring'
            WHEN EXTRACT(MONTH FROM o.order_purchase_timestamp)
                IN (6, 7, 8) THEN 'Summer'
            WHEN EXTRACT(MONTH FROM o.order_purchase_timestamp)
                IN (9, 10, 11) THEN 'Autumn'
            ELSE 'Winter'
        END AS season,
        oi.price AS sales
    FROM amazon_brazil.orders o
    JOIN amazon_brazil.order_items oi
        ON o.order_id = oi.order_id
) AS seasonal_sales
GROUP BY season
ORDER BY total_sales DESC;
  
```

Total rows: 4 Query complete 00:00:00.385



```

CASE
    WHEN EXTRACT(MONTH FROM o.order_purchase_timestamp)
        IN (3, 4, 5) THEN 'Spring'
    WHEN EXTRACT(MONTH FROM o.order_purchase_timestamp)
        IN (6, 7, 8) THEN 'Summer'
    WHEN EXTRACT(MONTH FROM o.order_purchase_timestamp)
        IN (9, 10, 11) THEN 'Autumn'
    ELSE 'Winter'
END AS season,
oi.price AS sales
FROM amazon_brazil.orders o
JOIN amazon_brazil.order_items oi
    ON o.order_id = oi.order_id
) AS seasonal_sales
GROUP BY season
ORDER BY total_sales DESC;
  
```

Data Output

season	total_sales
Spring	4216721.34
Summer	4123036.42
Autumn	3465750.03
Winter	3248812.51

Total rows: 4 Query complete 00:00:00.385

KEY INSIGHT

Q1. The marketing team wants to compare the total sales between different seasons. Use a subquery to calculate total sales for each season (Spring, Summer, Autumn, Winter) based on order purchase dates, and display the results. Spring is in the months of March, April and May. Summer is from June to August and Autumn is between September and November and rest months are Winter.

This analysis compares total sales across Spring, Summer, Autumn, and Winter based on order purchase dates.

Q2. The inventory team is interested in identifying products that have sales volumes above the overall average.

Question

Q2. The inventory team is interested in identifying products that have sales volumes above the overall average. Write a query that uses a subquery to filter products with a total quantity sold above the average quantity.

```

-- Q2: The inventory team wants to identify products that have
-- sales volumes above the overall average.
-- This is a subquery to filter products where the total quantity sold
-- is greater than the average quantity sold across all products.
SELECT
    product_id,
    total_quantity_sold
FROM (
    SELECT
        product_id,
        COUNT(*) AS total_quantity_sold
    FROM amazon_brazil.order_items
    GROUP BY product_id
) AS product_sales
WHERE total_quantity_sold > (
    SELECT AVG(total_quantity_sold)
    FROM (
        SELECT
            product_id,
            COUNT(*) AS total_quantity_sold
        FROM amazon_brazil.order_items
        GROUP BY product_id
    ) AS average_sales
)
ORDER BY total_quantity_sold DESC;
  
```

Total rows: 6366 Query complete 00:00:00.126

product_id	total_quantity_sold
602e0700e1e798e0d4e6881466...	527
99e4788cd5248599e5c3ba53a23995...	488
422679e10f40062990d4242775e719...	484
385d110e8d3504a2311235e493d9...	392
388dc738842d70016e49c3917e37...	388
55796d0c66e033d87a070e1f1519f...	373
e1e427f60a877388e88edc7c6172a...	343
5310cd67ab7c41503e0d54ab772...	323
15e1e31e8f83233795e672e001...	281
3d0da7168ec895c781a6191c1e65...	274
21460988a8e180749452343e8...	260
7c1b0921ebd22470b9b9de975d83...	231
4d2425e0e0e0e1e1f1895e0e1a03...	226
5e848e4e52705440c0c07ab1c4b...	197
1b5072c23e4e0e0188a80197e545...	195
e0e4d0f2a3b4b5c54c258ae191...	194
42a2c92a6970b0c4c4e88ec7b...	183
e3e557da125f5a2c7e995d8f24...	182
5022340e466387c7b0591a1c0ba...	169

Total rows: 6366 Query complete 00:00:00.126

KEY INSIGHT

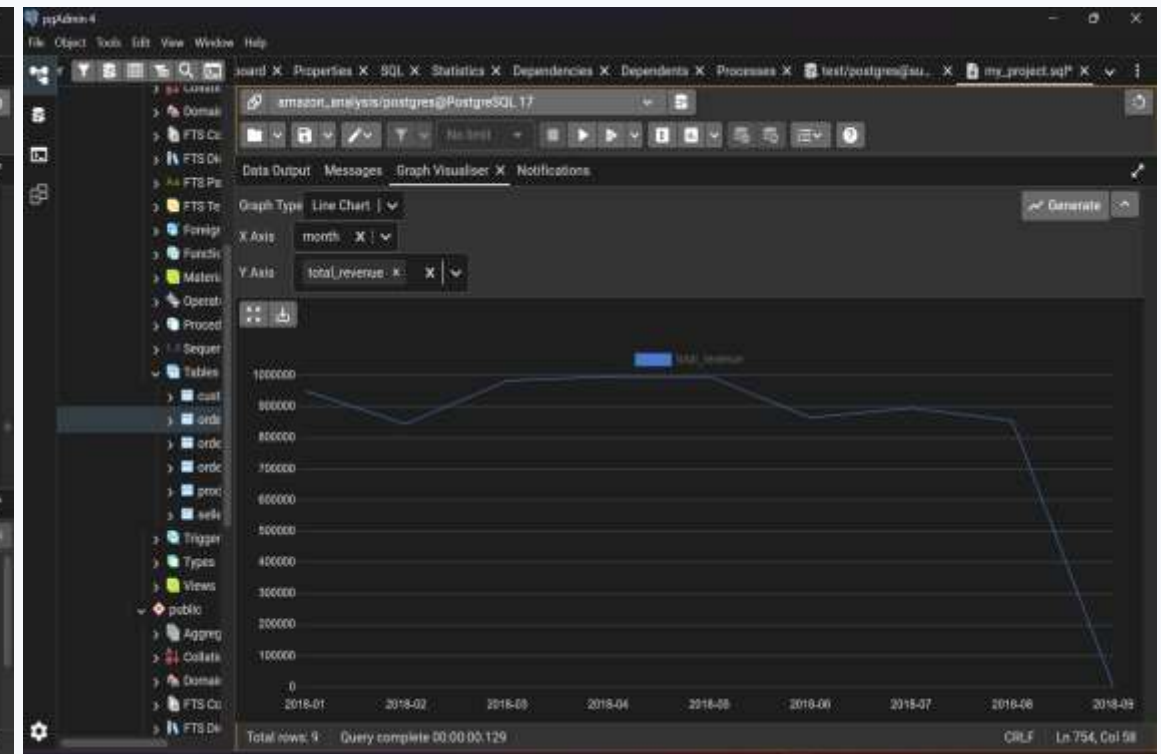
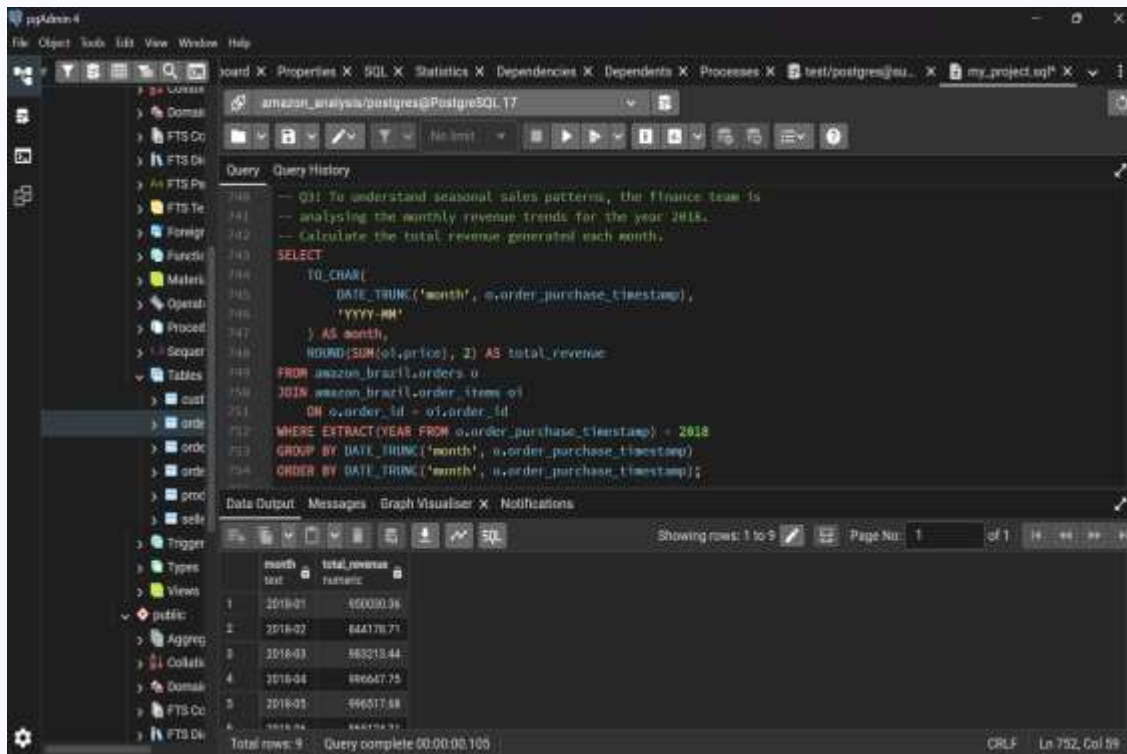
Q2. The inventory team is interested in identifying products that have sales volumes above the overall average. Write a query that uses a subquery to filter products with a total quantity sold above the average quantity.

This analysis identifies products whose total quantity sold is higher than the overall average sales quantity. These products can be considered better-performing products with stronger demand.

Q3. To understand seasonal sales patterns, the finance team is analysing the monthly revenue trends

Question

Q3. To understand seasonal sales patterns, the finance team is analysing the monthly revenue trends over the past year (year 2018). Run a query to calculate total revenue generated each month and identify periods of peak and low sales. Export the data to Excel and create a graph to visually represent revenue changes across the months.



KEY INSIGHT

Q3. To understand seasonal sales patterns, the finance team is analysing the monthly revenue trends over the past year (year 2018). Run a query to calculate total revenue generated each month and identify periods of peak and low sales. Export the data to Excel and create a graph to visually represent revenue changes across the months.

The analysis tracks monthly revenue trends for the year 2018, helping identify periods of peak and low sales. The Excel graph provides a clear visual representation of how revenue changed across different months.

Q4. A loyalty program is being designed for Amazon India. Create a segmentation based on purc

Question

Q4. A loyalty program is being designed for Amazon India. Create a segmentation based on purchase frequency: 'Occasional' for customers with 1-2 orders, 'Regular' for 3-5 orders, and 'Loyal' for more than 5 orders. Use a CTE to classify customers and their count and generate a chart in Excel to show the proportion of each segment.

The screenshot shows the pgAdmin 4 interface with a query editor. The query is as follows:

```

-- Q4: A loyalty program is being designed for Amazon India.
-- Create customer segments based on purchase frequency using a CTE.
-- Occasional: 1 to 2 orders
-- Regular: 3 to 5 orders
-- Loyal: More than 5 orders
WITH customer_orders AS (
    SELECT
        c.customer_unique_id,
        COUNT(DISTINCT o.order_id) AS total_orders
    FROM amazon_brazil.customers c
    JOIN amazon_brazil.orders o
    ON c.customer_id = o.customer_id
    GROUP BY c.customer_unique_id
),
customer_segments AS (
    SELECT
        customer_unique_id,
        CASE
            WHEN total_orders BETWEEN 1 AND 2 THEN 'Occasional'
            WHEN total_orders BETWEEN 3 AND 5 THEN 'Regular'
            WHEN total_orders > 5 THEN 'Loyal'
        END AS customer_type
    FROM customer_orders
)
SELECT

```

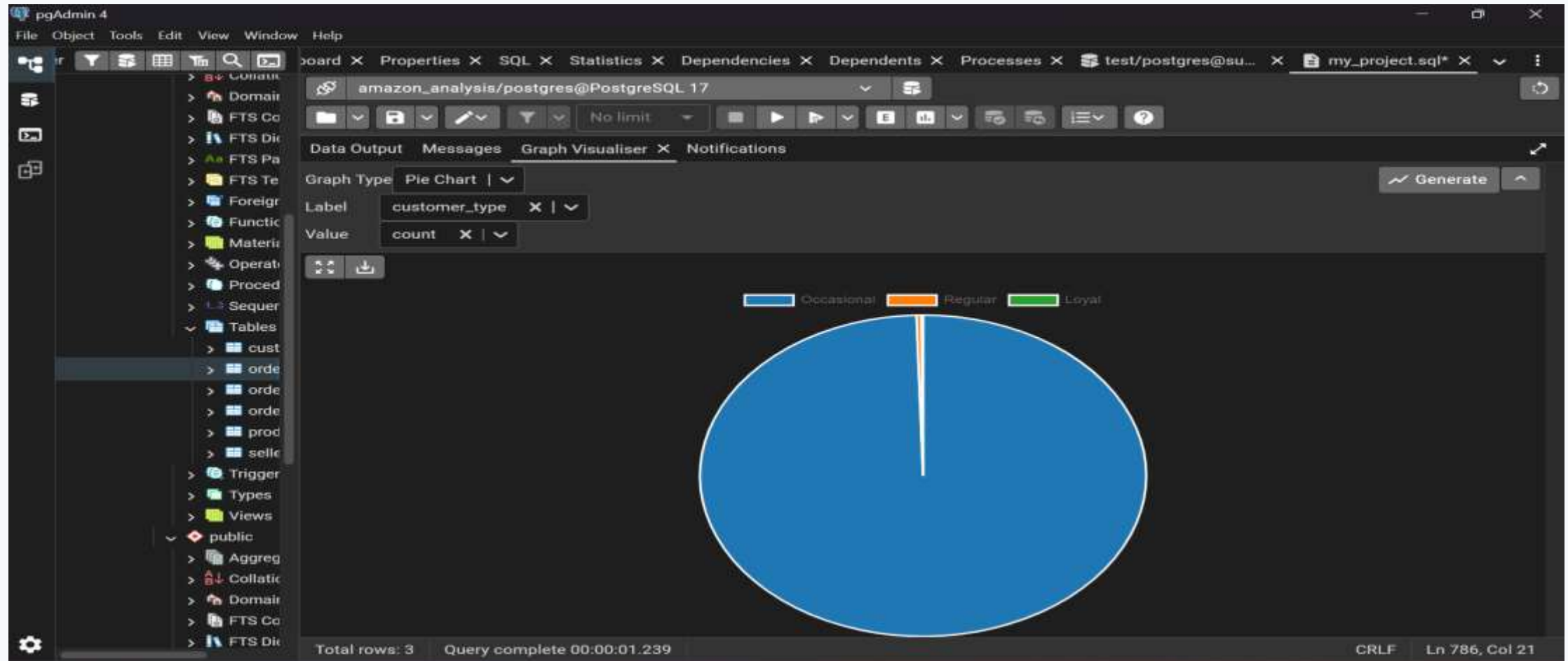
The status bar at the bottom indicates: Total rows: 3 Query complete 00:00:01.249

The screenshot shows the same pgAdmin 4 interface, but now displaying the results of the query in the 'Data Output' tab. The results are as follows:

customer_type	count
Occasional	94636
Regular	323
Loyal	106

The status bar at the bottom indicates: Total rows: 3 Query complete 00:00:01.249

Q4. A loyalty program is being designed for Amazon India. Create a segmentation based on purch



KEY INSIGHT

Q4. A loyalty program is being designed for Amazon India. Create a segmentation based on purchase frequency: 'Occasional' for customers with 1-2 orders, 'Regular' for 3-5 orders, and 'Loyal' for more than 5 orders. Use a CTE to classify customers and their count and generate a chart in Excel to show the proportion of each segment.

This analysis segments customers into Occasional, Regular, and Loyal groups based on their purchase frequency.

Q5. Amazon wants to identify high-value customers to target for an exclusive rewards program.

Question

Q5. Amazon wants to identify high-value customers to target for an exclusive rewards program. You are required to rank customers based on their average order value (avg_order_value) to find the top 20 customers.

The screenshot shows the pgAdmin 4 interface with a SQL query editor. The query is as follows:

```

-- Q5! Amazon wants to identify high-value customers for an
-- exclusive rewards program.
-- Calculate the average order value for each customer,
-- rank customers from highest to lowest, and display the top 20.

WITH customer_order_values AS (
    SELECT
        a.customer_id,
        a.order_id,
        SUM(op.payment_value) AS order_value
    FROM amazon_brazil.orders o
    JOIN amazon_brazil.order_payments op
    ON o.order_id = op.order_id
    GROUP BY
        a.customer_id,
        a.order_id
),
customer_avg AS (
    SELECT
        customer_id,
        AVG(order_value) AS avg_order_value
    FROM customer_order_values
    GROUP BY customer_id
)
SELECT
    customer_id,
    ROUND(avg_order_value, 2) AS avg_order_value,

```

Total rows: 20 Query complete 00:00:01.063

The screenshot shows the pgAdmin 4 interface with the same SQL query editor. The query is as follows:

```

SELECT
    customer_id,
    ROUND(avg_order_value, 2) AS avg_order_value,
    RANK() OVER (
        ORDER BY avg_order_value DESC
    ) AS customer_rank
FROM customer_avg
ORDER BY customer_rank
LIMIT 20;

```

The Data Output tab shows the results of the query:

customer_id	avg_order_value	customer_rank
1817b130775626295a34a6341a47e	12664.88	1
4c5b2b4d4e574942386b71631fa01	7274.88	2
c9e2731c3b381845f6800a7601a4	6629.31	3
f486464a0a0a338c325f16a91a	6022.21	4
3f6e775bce06a357f0eb04a47c	6726.88	5
55455da7c00713d150a7a6a972	6081.34	6
a755c1481476a6c3a677131294c24	4950.34	7
a8a2418220e9a476c1a0811a7	4809.44	8
248e59c022e15559a7a94a4a4a	4764.34	9
3e979669696322a0418a34a61a	4681.78	10

Total rows: 20 Query complete 00:00:01.063

KEY INSIGHT

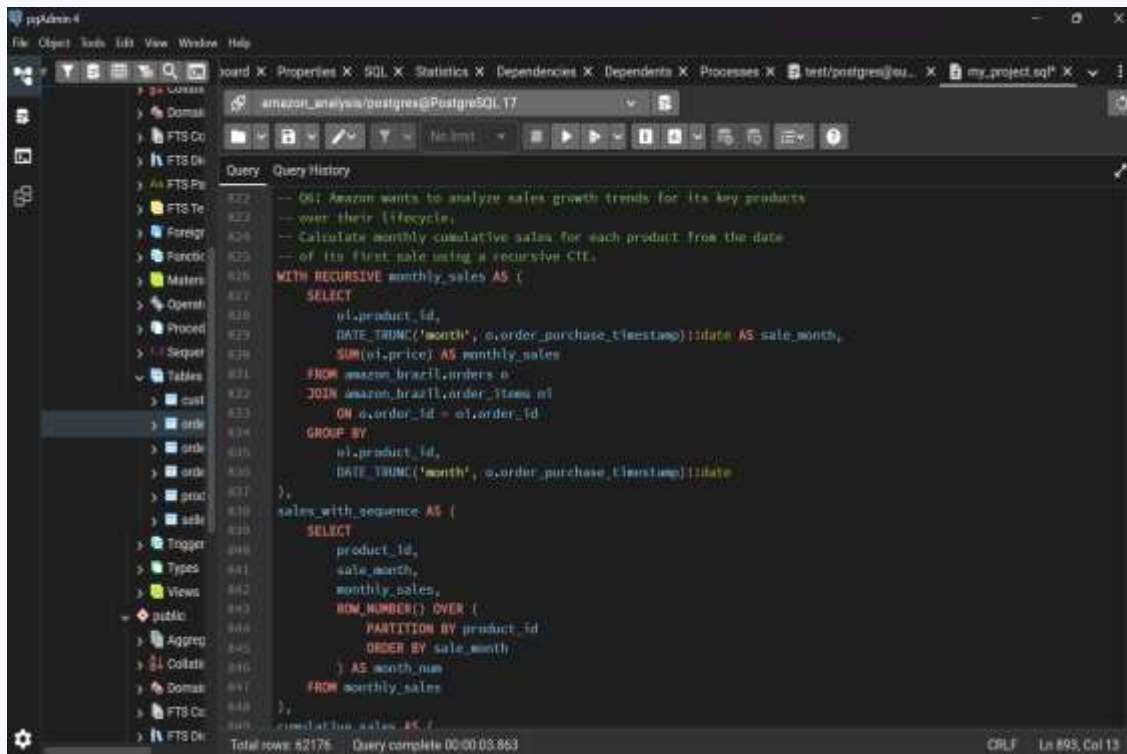
Q5. Amazon wants to identify high-value customers to target for an exclusive rewards program. You are required to rank customers based on their average order value (avg_order_value) to find the top 20 customers.

This analysis identifies the top 20 high-value customers by ranking them based on their average order value.

Q6. Amazon wants to analyze sales growth trends for its key products over their lifecycle. Calcul

Question

Q6. Amazon wants to analyze sales growth trends for its key products over their lifecycle. Calculate monthly cumulative sales for each product from the date of its first sale. Use a recursive CTE to compute the cumulative sales (total_sales) for each product month by month.

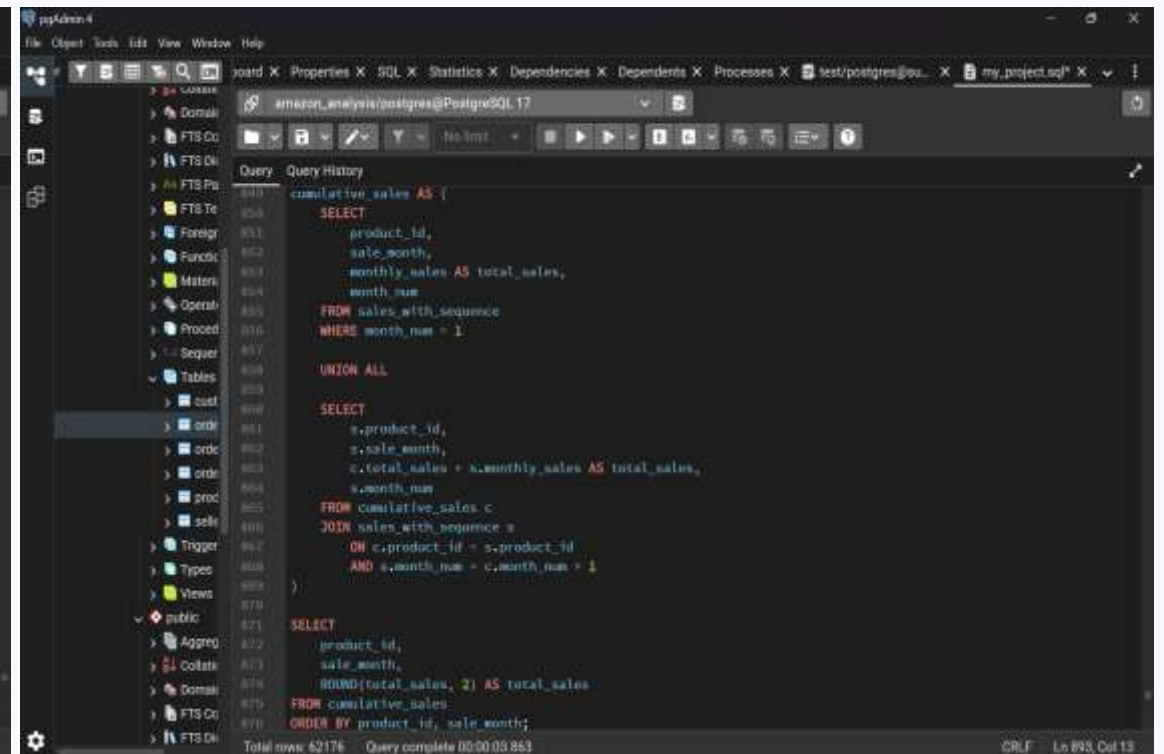


```

-- Q6: Amazon wants to analyze sales growth trends for its key products
-- over their lifecycle.
-- Calculate monthly cumulative sales for each product from the date
-- of its first sale using a recursive CTE.
WITH RECURSIVE monthly_sales AS (
    SELECT
        oi.product_id,
        DATE_TRUNC('month', o.order_purchase_timestamp)::date AS sale_month,
        SUM(oi.price) AS monthly_sales
    FROM amazon_brazil.orders o
    JOIN amazon_brazil.order_items oi
    ON o.order_id = oi.order_id
    GROUP BY
        oi.product_id,
        DATE_TRUNC('month', o.order_purchase_timestamp)::date
),
sales_with_sequence AS (
    SELECT
        product_id,
        sale_month,
        monthly_sales,
        ROW_NUMBER() OVER (
            PARTITION BY product_id
            ORDER BY sale_month
        ) AS month_num
    FROM monthly_sales
),
cumulative_sales AS (
    SELECT
        product_id,
        sale_month,
        monthly_sales AS total_sales,
        month_num
    FROM sales_with_sequence
    WHERE month_num = 1
    UNION ALL
    SELECT
        s.product_id,
        s.sale_month,
        c.total_sales + s.monthly_sales AS total_sales,
        s.month_num
    FROM cumulative_sales c
    JOIN sales_with_sequence s
    ON c.product_id = s.product_id
    AND s.month_num = c.month_num + 1
)
SELECT
    product_id,
    sale_month,
    ROUND(total_sales, 2) AS total_sales
FROM cumulative_sales
ORDER BY product_id, sale_month;

```

Total rows: 62176 Query complete 00:00:03.863



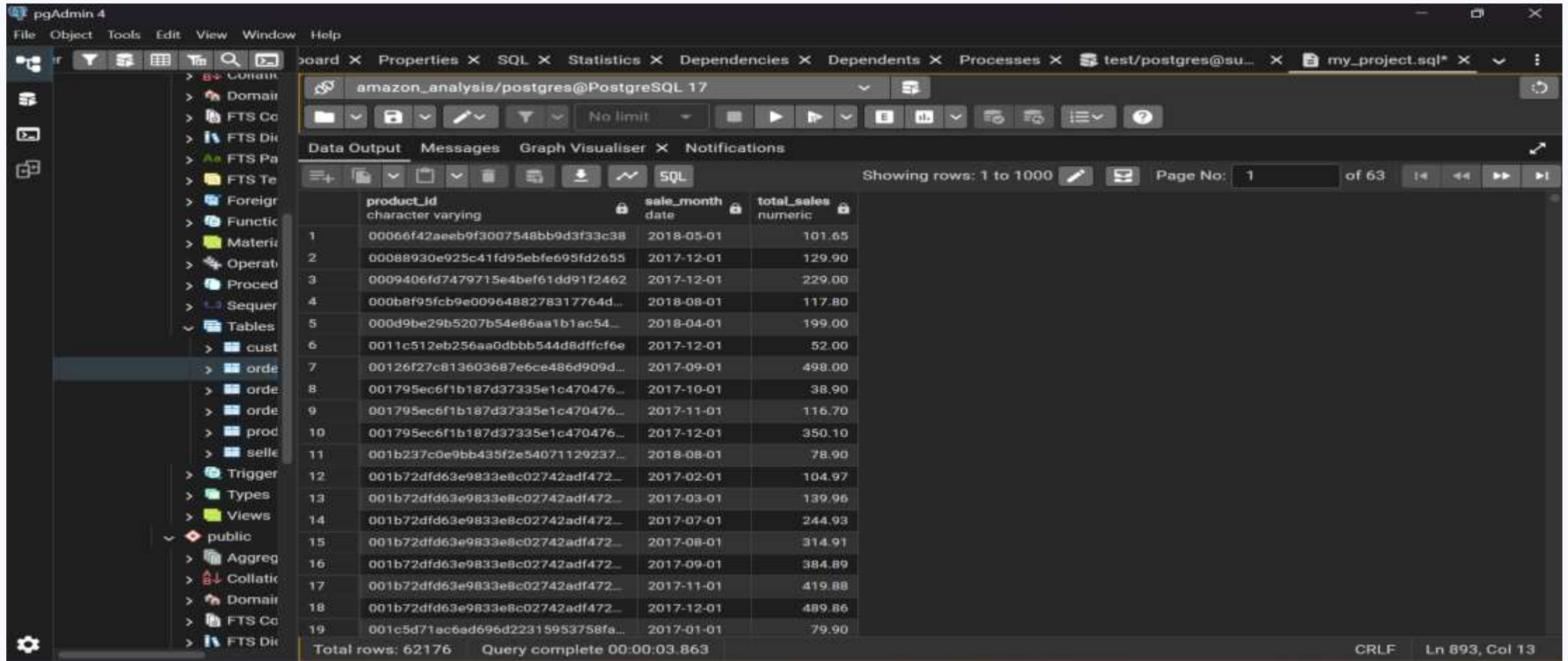
```

-- Q6: Amazon wants to analyze sales growth trends for its key products
-- over their lifecycle.
-- Calculate monthly cumulative sales for each product from the date
-- of its first sale using a recursive CTE.
WITH RECURSIVE monthly_sales AS (
    SELECT
        oi.product_id,
        DATE_TRUNC('month', o.order_purchase_timestamp)::date AS sale_month,
        SUM(oi.price) AS monthly_sales
    FROM amazon_brazil.orders o
    JOIN amazon_brazil.order_items oi
    ON o.order_id = oi.order_id
    GROUP BY
        oi.product_id,
        DATE_TRUNC('month', o.order_purchase_timestamp)::date
),
sales_with_sequence AS (
    SELECT
        product_id,
        sale_month,
        monthly_sales,
        ROW_NUMBER() OVER (
            PARTITION BY product_id
            ORDER BY sale_month
        ) AS month_num
    FROM monthly_sales
),
cumulative_sales AS (
    SELECT
        product_id,
        sale_month,
        monthly_sales AS total_sales,
        month_num
    FROM sales_with_sequence
    WHERE month_num = 1
    UNION ALL
    SELECT
        s.product_id,
        s.sale_month,
        c.total_sales + s.monthly_sales AS total_sales,
        s.month_num
    FROM cumulative_sales c
    JOIN sales_with_sequence s
    ON c.product_id = s.product_id
    AND s.month_num = c.month_num + 1
)
SELECT
    product_id,
    sale_month,
    ROUND(total_sales, 2) AS total_sales
FROM cumulative_sales
ORDER BY product_id, sale_month;

```

Total rows: 62176 Query complete 00:00:03.863

Q6. Amazon wants to analyze sales growth trends for its key products over their lifecycle. Calcul



The screenshot shows the pgAdmin 4 interface with a query result displayed. The query is executed against the 'amazon_analysis/postgres@PostgreSQL 17' database. The result shows 19 rows of data, with columns: product_id, sale_month, and total_sales. The status bar indicates 'Total rows: 62176' and 'Query complete 00:00:03.863'.

	product_id	sale_month	total_sales
1	00066f42aeeb9f3007548bb9d3f33c38	2018-05-01	101.65
2	00088930e925c41fd95ebfe695fd2655	2017-12-01	129.90
3	0009406fd7479715e4bef61dd91f2462	2017-12-01	229.00
4	000b8f95fcb9e0096488278317764d...	2018-08-01	117.80
5	000d9be29b5207b54e86aa1b1ac54...	2018-04-01	199.00
6	0011c512eb256aa0dbbb544d8dffcf6e	2017-12-01	52.00
7	00126f27c813603687e6ce486d909d...	2017-09-01	498.00
8	001795ec6f1b187d37335e1c470476...	2017-10-01	38.90
9	001795ec6f1b187d37335e1c470476...	2017-11-01	116.70
10	001795ec6f1b187d37335e1c470476...	2017-12-01	350.10
11	001b237c0e9bb435f2e54071129237...	2018-08-01	78.90
12	001b72dfd63e9833e8c02742adf472...	2017-02-01	104.97
13	001b72dfd63e9833e8c02742adf472...	2017-03-01	139.96
14	001b72dfd63e9833e8c02742adf472...	2017-07-01	244.93
15	001b72dfd63e9833e8c02742adf472...	2017-08-01	314.91
16	001b72dfd63e9833e8c02742adf472...	2017-09-01	384.89
17	001b72dfd63e9833e8c02742adf472...	2017-11-01	419.88
18	001b72dfd63e9833e8c02742adf472...	2017-12-01	489.86
19	001c5d71ac6ad696d22315953758fa...	2017-01-01	79.90

KEY INSIGHT

Q6. Amazon wants to analyze sales growth trends for its key products over their lifecycle. Calculate monthly cumulative sales for each product from the date of its first sale. Use a recursive CTE to compute the cumulative sales (total_sales) for each product month by month.

This analysis tracks the monthly cumulative sales of each product from its first sale onward. The cumulative sales trend helps Amazon India understand how product sales grow over time and identify products with strong or slow sales performance.

Q7. To understand how different payment methods affect monthly sales growth, Amazon wants

Question

Q7. To understand how different payment methods affect monthly sales growth, Amazon wants to compute the total sales for each payment method and calculate the month-over-month growth rate for the past year (year 2018). Write query to first calculate total monthly sales for each payment method, then compute the percentage change from the previous month.

```

-- Q7: To understand how different payment methods affect monthly
-- sales growth, calculate the total monthly sales for each payment
-- method and the percentage change from the previous month for 2018.
WITH monthly_sales AS (
    SELECT
        op.payment_type,
        DATE_TRUNC(
            'month',
            o.order_purchase_timestamp
       )::date AS sale_month,
        SUM(op.payment_value) AS monthly_total
    FROM amazon_brazil.orders o
    JOIN amazon_brazil.order_payments op
        ON o.order_id = op.order_id
    WHERE EXTRACT(YEAR FROM o.order_purchase_timestamp) = 2018
    GROUP BY
        op.payment_type,
        DATE_TRUNC('month', o.order_purchase_timestamp)::date
),
sales_with_previous AS (
    SELECT
        payment_type,
        sale_month,
        monthly_total,
        LAG(monthly_total) OVER (
            PARTITION BY payment_type
            ORDER BY sale_month
        ) AS previous_month_total
    FROM monthly_sales
)
SELECT
    payment_type,
    sale_month,
    ROUND(monthly_total, 2) AS monthly_total,
    ROUND(
        (monthly_total - previous_month_total)
        / NULLIF(previous_month_total, 0)
        * 100
    ) AS monthly_change
FROM sales_with_previous
ORDER BY payment_type, sale_month;

```

Total rows: 36 Query complete 00:00:00.119

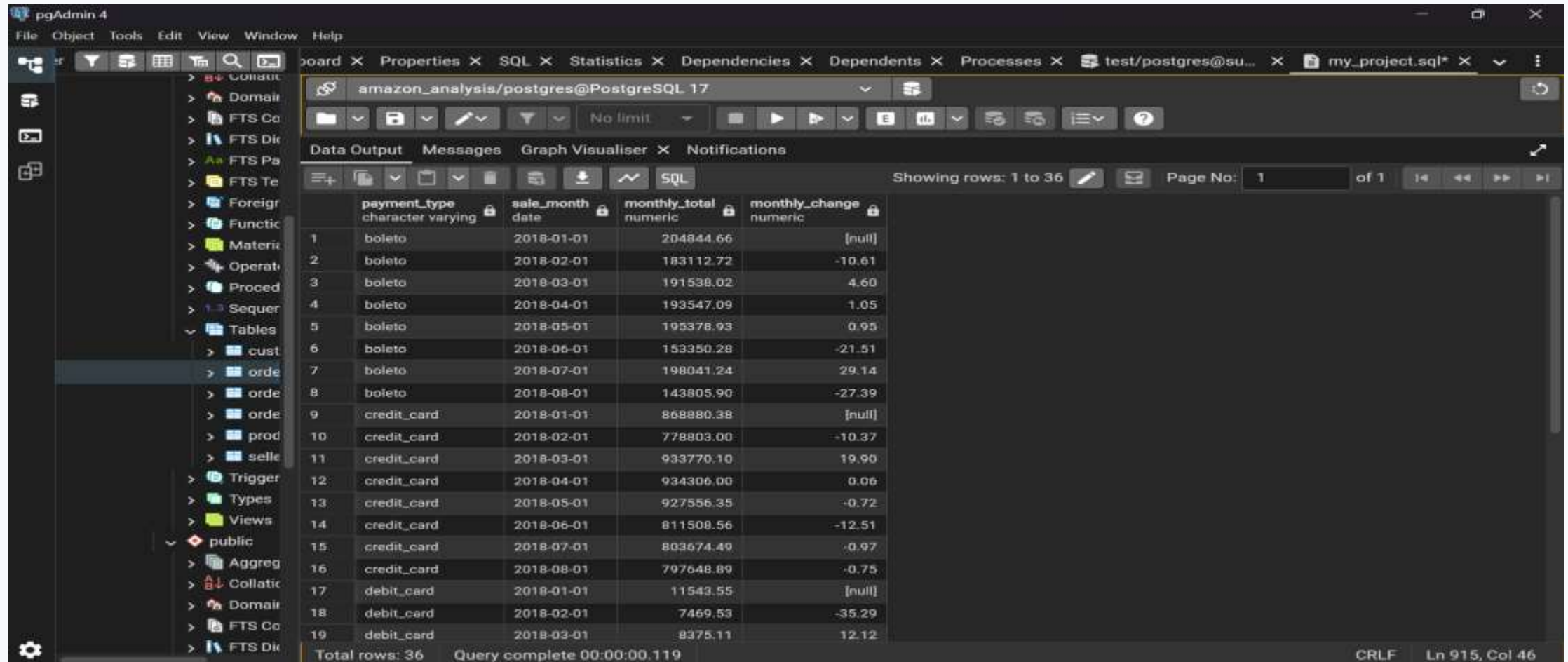
```

-- Q7: To understand how different payment methods affect monthly
-- sales growth, calculate the total monthly sales for each payment
-- method and the percentage change from the previous month for 2018.
WITH monthly_sales AS (
    SELECT
        op.payment_type,
        DATE_TRUNC(
            'month',
            o.order_purchase_timestamp
       )::date AS sale_month,
        SUM(op.payment_value) AS monthly_total
    FROM amazon_brazil.orders o
    JOIN amazon_brazil.order_payments op
        ON o.order_id = op.order_id
    WHERE EXTRACT(YEAR FROM o.order_purchase_timestamp) = 2018
    GROUP BY
        op.payment_type,
        DATE_TRUNC('month', o.order_purchase_timestamp)::date
),
sales_with_previous AS (
    SELECT
        payment_type,
        sale_month,
        monthly_total,
        LAG(monthly_total) OVER (
            PARTITION BY payment_type
            ORDER BY sale_month
        ) AS previous_month_total
    FROM monthly_sales
)
SELECT
    payment_type,
    sale_month,
    ROUND(monthly_total, 2) AS monthly_total,
    ROUND(
        (monthly_total - previous_month_total)
        / NULLIF(previous_month_total, 0)
        * 100
    ) AS monthly_change
FROM sales_with_previous
ORDER BY payment_type, sale_month;

```

Total rows: 36 Query complete 00:00:00.119

Q7. To understand how different payment methods affect monthly sales growth, Amazon wants



The screenshot shows the pgAdmin 4 interface with a query result displayed. The query is executed against the 'amazon_analysis' database. The result shows monthly sales data for the year 2018, categorized by payment method. The columns are: payment_type, sale_month, monthly_total, and monthly_change. The data is sorted by sale_month in ascending order.

	payment_type	sale_month	monthly_total	monthly_change
1	boleto	2018-01-01	204844.66	[null]
2	boleto	2018-02-01	183112.72	-10.61
3	boleto	2018-03-01	191538.02	4.60
4	boleto	2018-04-01	193547.09	1.05
5	boleto	2018-05-01	195378.93	0.95
6	boleto	2018-06-01	153350.28	-21.51
7	boleto	2018-07-01	198041.24	29.14
8	boleto	2018-08-01	143805.90	-27.39
9	credit_card	2018-01-01	868880.38	[null]
10	credit_card	2018-02-01	778803.00	-10.37
11	credit_card	2018-03-01	933770.10	19.90
12	credit_card	2018-04-01	934306.00	0.06
13	credit_card	2018-05-01	927556.35	-0.72
14	credit_card	2018-06-01	811508.56	-12.51
15	credit_card	2018-07-01	803674.49	-0.97
16	credit_card	2018-08-01	707648.89	-0.75
17	debit_card	2018-01-01	11543.55	[null]
18	debit_card	2018-02-01	7469.53	-35.29
19	debit_card	2018-03-01	8375.11	12.12

Total rows: 36 Query complete 00:00:00.119 CRLF Ln 915, Col 46

KEY INSIGHT

Q7. To understand how different payment methods affect monthly sales growth, Amazon wants to compute the total sales for each payment method and calculate the month-over-month growth rate for the past year (year 2018). Write query to first calculate total monthly sales for each payment method, then compute the percentage change from the previous month.

This analysis calculates the monthly total sales for each payment method in 2018 and measures the month-over-month growth or decline compared with the previous month

KEY FINDINGS & RECOMMENDATIONS

- Use seasonal and monthly sales trends for inventory and marketing planning.
- Target Returning and Loyal customers with personalized offers and loyalty programs.
- Prioritize high-revenue and above-average-selling products.
- Monitor payment-method behavior and monthly growth trends.
- Improve data quality by correcting missing or incomplete product categories.

THANK YOU

Amazon Brazil E-Commerce Data Analysis